

PHOENIX BACKEND PROJECT

Notification Service — Member 4 Final Report

Observability, Documentation, and Quality Assurance

Project	Phoenix Backend
Component	Notification Service
Role	Member 4 — Observability, Documentation, and Quality Assurance
Technology	TypeScript, Node.js, Express, RabbitMQ, Supabase/PostgreSQL
Report purpose	Document Member 4 analysis, implementation, testing, evidence, and QA findings

1. Introduction

The Notification Service is responsible for supporting notification delivery and retrieval within the Phoenix backend. The Member 4 role which assigned for me focuses on observability, documentation, quality assurance, health checks, logging, API documentation, and combined testing.

The assigned Member 4 outputs are a test plan, failure-scenario matrix, logging and monitoring requirements, Swagger checklist, health-check proposal, end-to-end testing flow, and a definition of completion. The implementation responsibilities include structured logging, health reporting, Swagger, test scaffolding, integration testing support, end-to-end evidence, and test-result documentation.

2. Member 4 Responsibilities

- Define logging and monitoring requirements for operational visibility.
- Define notification-service health-check requirements.
- Identify notification-service metrics and RabbitMQ consumer monitoring requirements.
- Identify error, failure, recovery, invalid-message, duplicate-message, and dependency-failure scenarios.
- Define and verify Swagger/OpenAPI documentation requirements.
- Define unit-testing and integration-testing requirements.
- Coordinate manual integration and end-to-end testing.
- Document test data, evidence, results, defects, and follow-up actions.

These responsibilities are based on the Notification Service task specification, which assigns Member 4 the Observability, Documentation, and Quality Assurance component.

3. Analysis and QA Design

3.1 Logging and Monitoring Requirements

- Use the shared logging approach for service startup, shutdown, health responses, errors, and important processing events.
- Record RabbitMQ connection status and consumer startup.
- Record message receipt and successful processing.
- Record processing failures and invalid-message handling.
- Avoid logging passwords, tokens, database credentials, or other secrets.
- Monitor RabbitMQ consumer availability and dependency failures.

The task specification identifies logging requirements, health checks, notification-service metrics, RabbitMQ consumer monitoring, and failure scenarios as core Member 4 analysis responsibilities.

3.2 Health-Check Proposal

- Return HTTP 200 when the service and required dependencies are available.
- Return an appropriate unhealthy/degraded status when a required dependency is unavailable.
- Include RabbitMQ availability in health reporting where RabbitMQ is required for normal message processing.
- Keep health responses simple and suitable for Docker/service monitoring.

The proposed design follows the analysis requirement that health reporting should identify service and relevant dependency availability.

3.3 Swagger Checklist

Swagger item	Requirement
HTTP method and endpoint/path	Required
Purpose and description	Required
Authentication requirements	Required where applicable
Request parameters/body	Required where applicable
Successful response and schema	Required
Relevant HTTP status codes	Required
Error responses	Required
Examples	Recommended where useful

The approved REST endpoints should be documented through Swagger and updated if the final API structure changes.

3.4 End-to-End Testing Flow



4. Member 4 Implementation Completed

- Structured logging was added to the Notification Service handlers and application startup/shutdown paths.
- Notification health and notification retrieval responses were logged through the shared logger.
- Notification handler failures return a controlled internal-server-error response rather than exposing internal error details.
- RabbitMQ connection status is tracked and reported by the Data Ingestion health check.
- RabbitMQ consumer startup, message receipt, success, failure, and invalid-message handling are logged.
- The API Gateway notification controller was updated to log dependency responses and handle gRPC errors safely.
- Swagger endpoints were verified through the API Gateway documentation.
- Jest and ts-jest test scaffolding was added for Notification Service unit tests.
- Six automated unit tests were executed successfully.
- Test-result documentation was created and updated with the observed QA findings.

The task specification explicitly includes structured logging, RabbitMQ consumer logs, error logs, health reporting, Swagger, test fixtures, integration setup, E2E scripts, and test-result documentation in the Member 4 implementation scope.

5. Test Plan and Test Results

Testing combined automated unit tests, service/API checks, Docker verification, RabbitMQ tests, authentication checks, Swagger verification, and failure/recovery scenarios.

ID	Test	Observed Result	Status
T01	Notification Service unit tests	6/6 Jest tests passed	PASS
T02	TypeScript project build	Build completed successfully	PASS
T03	Notification health endpoint	HTTP 200; service reported running	PASS
T04	Notification retrieval endpoint	HTTP 200; successful response returned	PASS
T05	Swagger/OpenAPI verification	Notification endpoints available and documented	PASS
T06	RabbitMQ unavailable health check	HTTP 503; dependency reported unavailable	PASS
T07	RabbitMQ recovery	HTTP 200 after Data Ingestion Service restart	PASS
T08	Invalid RabbitMQ message	Malformed JSON rejected and acknowledged/discarded	PASS
T09	Authentication without token	HTTP 401; No token provided	PASS
T10	Valid RabbitMQ hazard processing	Message received and processed successfully	PASS
T11	Duplicate RabbitMQ message	Same valid message processed twice; no idempotency prevention	FAIL / GAP
T12	Notification database retrieval	Endpoint currently returns a static response; DB retrieval not demonstrated	NOT VERIFIED
T13	Database failure scenario	Not fully verified during this testing session	NOT VERIFIED
T14	Automatic RabbitMQ consumer reconnection	Consumers required Data Ingestion Service restart after RabbitMQ recovery	GAP

6. Failure-Scenario Matrix

ID	Scenario	Expected	Observed	Status
F01	RabbitMQ unavailable	Health endpoint should identify dependency failure	HTTP 503 returned	PASS
F02	RabbitMQ restart	Consumers should recover or recovery should be identified	Consumers did not automatically recover; service restart restored them	GAP
F03	Malformed RabbitMQ message	Invalid message should not be processed as valid JSON	JSON parsing failed; message acknowledged/discarded	PASS
F04	Duplicate message	Duplicate event should not create unintended duplicate processing	Same message processed twice	FAIL / GAP
F05	Unauthenticated API request	Protected endpoint should reject request	HTTP 401 returned	PASS
F06	Database failure	Failure should be safely detected and logged	Not fully verified	NOT VERIFIED
F07	Notification retrieval	Endpoint should retrieve persisted notifications	Static response returned; DB retrieval not demonstrated	GAP

7. Test Evidence

Figure 1. Final Docker Compose service status

```
PS C:\Users\A3mus\Phoenix\backend> docker compose ps
```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
api-gateway	backend-api-gateway	"docker-entrypoint.s..."	api-gateway	7 minutes ago	Up 7 minutes	0.0.0.0:3001->3000/tcp
ingestion-service	backend-data-ingestion-service	"docker-entrypoint.s..."	data-ingestion-service	7 minutes ago	Up 7 minutes	0.0.0.0:50053->50053/tcp
notification-service	backend-notification-service	"docker-entrypoint.s..."	notification-service	7 minutes ago	Up 7 minutes	0.0.0.0:50052->50052/tcp
phoenix-rabbitmq	rabbitmq:4-management	"docker-entrypoint.s..."	rabbitmq	7 minutes ago	Up 7 minutes (healthy)	0.0.0.0:5672->5672/tcp, 0.0.0.0:15672->15672/tcp
phoenix-redis	redis:7	"docker-entrypoint.s..."	redis	7 minutes ago	Up 7 minutes (healthy)	0.0.0.0:6379->6379/tcp
storage-service	backend-storage-service	"docker-entrypoint.s..."	storage-service	7 minutes ago	Up 7 minutes	0.0.0.0:50054->50054/tcp
user-service	backend-user-service	"docker-entrypoint.s..."	user-service	7 minutes ago	Up 7 minutes	0.0.0.0:50051->50051/tcp

```
PS C:\Users\A3mus\Phoenix\backend>
```

<

Figure 2. Jest automated unit tests

```
PS C:\Users\A3mus\Phoenix\backend> npm test
> test
> jest --runInBand

PASS notification-service/src/grpc/notification.handler.test.ts
PASS notification-service/src/notification.service.test.ts

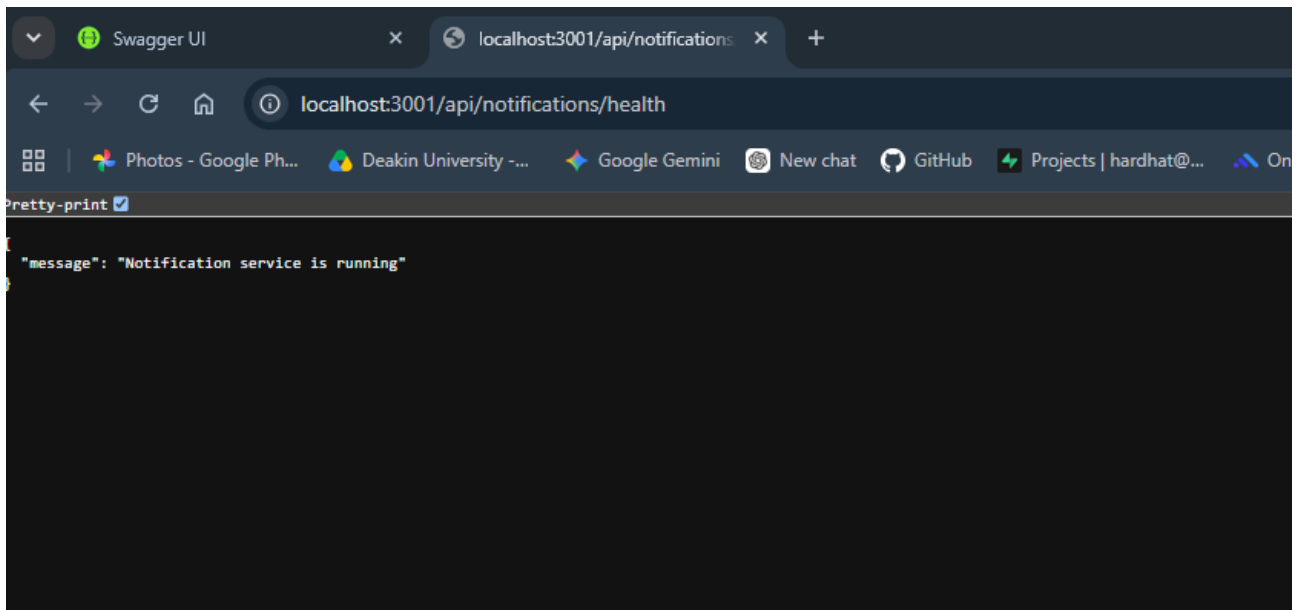
Test Suites: 2 passed, 2 total
Tests: 6 passed, 6 total
Snapshots: 0 total
Time: 3.413 s
Ran all test suites.
PS C:\Users\A3mus\Phoenix\backend>
```

Figure 3. TypeScript build

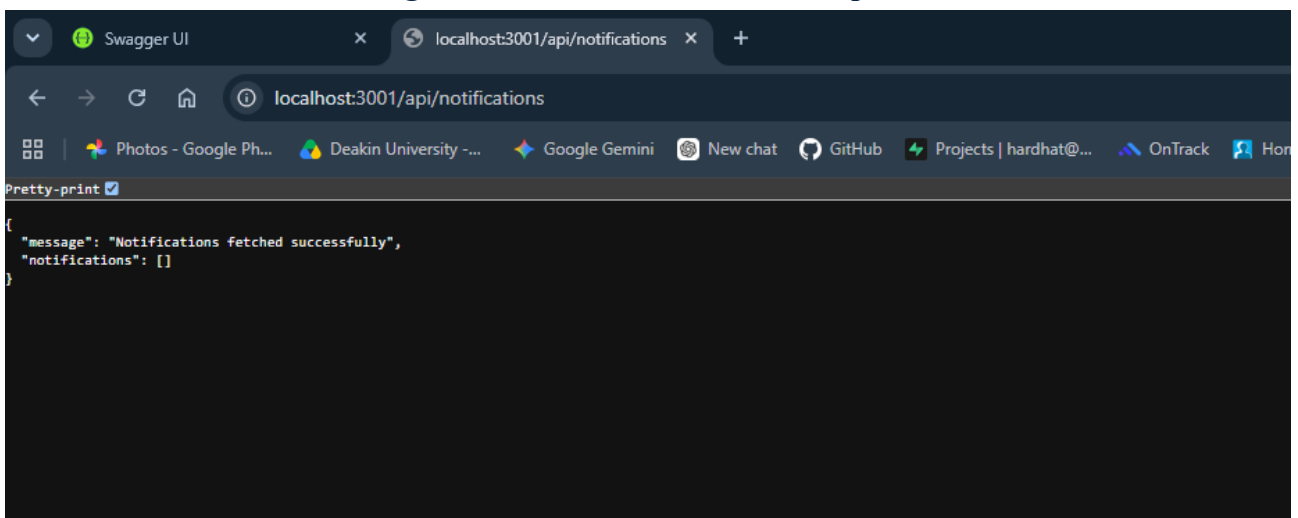
```
PS C:\Users\A3mus\Phoenix\backend> npm run build

> build
> tsc -b

PS C:\Users\A3mus\Phoenix\backend>
```

Figure 4. Notification health endpoint

The screenshot shows the Swagger UI interface in a web browser. The address bar displays `localhost:3001/api/notifications/health`. The response body, formatted with "Pretty-print", shows a JSON object: `{"message": "Notification service is running"}`. The browser's taskbar at the bottom includes icons for Photos, Deakin University, Google Gemini, New chat, GitHub, and Projects.

Figure 5. Notification retrieval endpoint

The screenshot shows the Swagger UI interface in a web browser. The address bar displays `localhost:3001/api/notifications`. The response body, formatted with "Pretty-print", shows a JSON object: `{"message": "Notifications fetched successfully", "notifications": []}`. The browser's taskbar at the bottom includes icons for Photos, Deakin University, Google Gemini, New chat, GitHub, Projects, OnTrack, and a user profile icon.

Figure 6. Swagger/OpenAPI verification

System

GET

/api/ingestion/health

Check ingestion service health

GET

/api/notifications/health

Check the health of the notification service

Returns the current operational status of the notification service.

Parameters

No parameters

Cancel

Execute

Clear

Responses

Curl

curl -X 'GET' \n'http://localhost:3001/api/notifications/health' \n-H 'accept: application/json'

Request URL

http://localhost:3001/api/notifications/health

Server response

Code	Details
200	Response body {\n "message": "Notification service is running"\n} Download Response headers access-control-allow-origin: *\ncontent-length: 45\ncontent-type: application/json; charset=utf-8\ndate: Tue, 15 Sep 2020 23:34:13 GMT\netag: W/"2c-9PR7vd0x1OLxfAp395SKrPp+NY8"\nx-powered-by: Express

Responses

Code	Description	Links
200	Notification service is running successfully Media type application/json Controls Accept header Example Value Schema {\n "message": "Notification service is healthy"\n}	No links
500	Failed to retrieve the notification service health Media type application/json Example Value Schema {\n "message": "Error fetching notification health"\n}	No links

Notifications

GET `/api/notifications` Retrieve all notifications

Retrieves the available notifications from the notification service.

Parameters Cancel

No parameters

Execute **Clear**

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:3001/api/notifications' \
  -H 'accept: application/json'
```

Request URL

`http://localhost:3001/api/notifications`

Server response

Code	Details
200	Response body <pre>{ "message": "Notifications fetched successfully", "notifications": [] }</pre> Response headers <pre>access-control-allow-origin: * connection: keep-alive content-length: 67 content-type: application/json; charset=utf-8 date: Tue, 15 Sep 2026 23:35:47 GMT etag: W/"43-gH+AE6630ViATGchJ7q4/AJCUY" keep-alive: timeout=5 x-powered-by: Express</pre>

Responses

Code	Description	Links
200	Notifications retrieved successfully Media type: application/json Controls Accept header. Example Value Schema <pre>{ "message": "Notifications retrieved successfully", "notifications": [{ "additionalProp1": {} }] }</pre>	No links
500	Failed to retrieve notifications Media type: application/json Example Value Schema <pre>{ "message": "Error fetching notifications" }</pre>	No links

Figure 7. RabbitMQ unavailable health check

```
PS C:\Users\3mus\Phoenix\backend> Invoke-WebRequest http://localhost:3001/api/ingestion/health
Invoke-WebRequest : {"status":503,"message":"Data ingestion service is unhealthy: RabbitMQ is unavailable"}
At line:1 char:1
+ Invoke-WebRequest http://localhost:3001/api/ingestion/health
+ ~~~~~
+ CategoryInfo          : InvalidOperation: (System.Net.HttpWebRequest:HttpWebRequest) [Invoke-WebRequest], WebException
+ FullyQualifiedErrorId : WebCmdletWebResponseException,Microsoft.PowerShell.Commands.InvokeWebRequestCommand

PS C:\Users\3mus\Phoenix\backend> curl.exe -i http://localhost:3001/api/ingestion/health
HTTP/1.1 503 Service Unavailable
X-Powered-By: Express
Access-Control-Allow-Origin: *
Content-Type: application/json; charset=utf-8
Content-Length: 87
ETag: W/"57-i26ee0AtxAM5ThNpnMg9cjYP/hY"
Date: Tue, 15 Sep 2026 23:41:29 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"status":503,"message":"Data ingestion service is unhealthy: RabbitMQ is unavailable"}
PS C:\Users\3mus\Phoenix\backend>
```

Figure 8. Invalid RabbitMQ message handling

[illegible]

Figure 9. Authentication test

```
PS C:\Users\A3mus\Phoenix\backend> curl.exe -i -X POST http://localhost:3001/api/storage/upload
HTTP/1.1 401 Unauthorized
X-Powered-By: Express
Access-Control-Allow-Origin: *
Content-Type: application/json; charset=utf-8
Content-Length: 44
ETag: W/"2c-FtsHYt+khUOV9W9+yz/wLxZoogE"
Date: Tue, 15 Sep 2026 23:50:11 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"status":401,"message":"No token provided"}
PS C:\Users\A3mus\Phoenix\backend>
```

Figure 10. Valid RabbitMQ hazard processing

The screenshot displays the RabbitMQ management interface with a queue named 'hazard_data_creation_queue'. The queue settings show a type of 'classic' and a message size limit of 1024. The 'Messages' tab shows 0 messages in the queue. The 'Windows PowerShell' terminal window shows the output of a 'docker compose logs' command for the 'data-ingestion-service'. The logs indicate that the service is running successfully, with messages being received and processed. The logs also show that the service is connected to the RabbitMQ broker and is able to consume messages from the 'hazard_data_creation_queue'.

8. Automated Unit Testing

Jest, ts-jest, and Jest type definitions were added to the backend test setup. Two Notification Service test suites were created: service-level tests and gRPC-handler tests.

- Notification service health response test — PASS.
- Notification service retrieval response test — PASS.
- gRPC health handler success test — PASS.
- gRPC notification handler success test — PASS.
- gRPC health handler error-path test — PASS.
- gRPC notification handler error-path test — PASS.

Automated result: 2 test suites passed; 6 tests passed; 0 failed.

9. Swagger/OpenAPI Verification

- Notification health endpoint was verified through the API documentation.
- Notification retrieval endpoint was verified through the API documentation/API Gateway.
- Endpoint response behaviour was checked against the documented HTTP responses.
- Authentication protection was separately tested on a protected API endpoint.

10. Observability and Health-Check Findings

- Notification Service startup and operational events use the shared logging approach.
- Notification handler errors are logged without returning internal exception details to the API caller.
- RabbitMQ connection state is reflected in the Data Ingestion Service health check.
- When RabbitMQ was stopped, the health endpoint returned HTTP 503.
- After RabbitMQ was restarted, the Data Ingestion Service required a restart to restore its consumers.
- This identifies automatic RabbitMQ reconnection as a reliability improvement area.

11. Remaining QA Gaps and Recommendations

QA item	Follow-up
Duplicate-event/idempotency handling	The same valid RabbitMQ message was processed twice. A team-level event ID/idempotency design is required.
Automatic RabbitMQ reconnection	Consumers did not automatically recover after RabbitMQ returned. Reconnection and consumer re-registration should be considered.
Notification database retrieval	The current notification retrieval response is static. Actual database retrieval should be implemented and verified.
Database failure testing	A controlled database-unavailable test should be completed when a safe test environment is available.
Complete integration/E2E automation	Manual integration and E2E evidence was collected, but a complete automated integration/E2E suite remains future work.
Shared fixtures/mock publisher	Reusable fixtures and a mock event publisher were part of the Member 4 design but were not fully demonstrated in the current test session.

12. Definition of Completion

From the Member 4 QA perspective, completion requires successful execution and evidence of the agreed functional, observability, documentation, reliability, and testing requirements. The task specification identifies successful RabbitMQ consumption, safe invalid-event handling, duplicate-event prevention, authentication/authorisation, health checks, structured logging, Swagger/OpenAPI, and unit/integration testing as acceptance criteria.

Completion criterion	Current status
Notification service starts successfully	Completed
Health endpoint responds correctly	Completed
Structured logging implemented for important events	Completed for tested Notification Service paths
RabbitMQ consumer activity logged	Completed
RabbitMQ failure behaviour tested	Completed
Invalid message handling tested	Completed
Authentication protection tested	Completed
Swagger/OpenAPI verified	Completed
Unit tests implemented and passing	Completed — 6/6
Duplicate messages prevented	Not completed — gap identified
Automatic RabbitMQ consumer recovery	Not completed — gap identified
Notification database retrieval verified	Not completed — current response is static
Database failure handling verified	Not completed
Complete automated integration/E2E coverage	Not completed

13. Conclusion

The Member 4 observability, documentation, and quality-assurance work established a documented test approach, structured logging improvements, health-check verification, Swagger verification, automated unit tests, RabbitMQ failure testing, authentication testing, invalid-message handling, and successful RabbitMQ message-processing evidence.

The testing also identified important implementation gaps rather than reporting all scenarios as successful. Duplicate message processing, automatic RabbitMQ consumer recovery, actual notification database retrieval, and complete database failure testing remain follow-up items. These findings should be communicated to the relevant component owners and tracked as part of the team's remaining implementation work.